CS3210

AY25/26 Sem 2

by ngmh

Introduction

- In recent times, processors have gained more logical cores
- von Neumann Model: Each core is an independent processing unit which can execute a linear stream of instructions
- Parallelism is limited by dependencies and communication
- Speedup might not be linear
- Concurrency: Multiple tasks make progress at the same time
- Parallelism: Tasks actually execute simultaneously
- Decomposition: Splitting problem into smaller tasks
- Scheduling and Mapping: Defining when a task should run and on which PU
- Performance: Execution Time (Latency) v/s Throughput
- Parallel Execution Time consists of computation time and parallelisation overheads
- Examples of overhead include distribution of work, synchronisation, information exchange, memory issues, idle time

Processes, Threads, Synchronisation

- Steps: Decomposition, Scheduling (tasks to processes or threads), and Mapping (processes or threads to physical processors)
- Process: Instance of a program in execution; consists of executable, global data, stack and heap, registers, own address space
- IPC: Shared memory, Message Passing e.g. Pipes and Signal
- Processes are costly to create and communicate between
- Threads are lightweight extensions of processes; consisting of PC, SP, and registers; while sharing the same address space (data and heap)
- Threads are faster to generate
- Threads can be either User level (managed by library, invisible to OS) or Kernel threads (managed by OS)
- Can be many-to-one (all mapped to one process), one-to-one (one use thread mapped to one kernel thread), or many-to-many (use thread mapped to set of kernel threads which OS schedules)
- Synchronisation: Coordinating access to shared resources
- Mechanisms include locks, mutexes, semaphores, monitors, condition variables
- Race Condition: Multiple execution paths execute at the same time and finish in a different order than expected
- Critical race conditions cause invalid execution and software bugs
- Critical Section protects parts of the program by allowing one thread at a time
- Mutexes protect access to shared resources through acquire and release operations
- Data Race: Two concurrent threads access a shared resource without any protection and at least one thread modifies the shared resource
- Semaphores provide mutual exclusion through atomic counters
- Deadlock: Every process is waiting for an event that can be caused only by another process in the set
- Coffman Conditions: Mutual exclusion, Hold and wait, No pre-emption, Circular wait
- Can ignore, prevent, avoid, or detect and recover
- Starvation: Process is prevented from making progress because some other process has the resource it requires; side effect of scheduling algorithm
- Livelock: States of processes constantly change but none make progress
- Lock Implementation: Test and Set atomic instruction

Parallel Computing Architectures

- Bit Level Parallelism: Increase data processed per second via increasing word size (unit of transfer) since PU is optimised for operations on word sized values
- Superword Level Parallelism: Same word size, but larger registers, process more data with one instruction
- Tradeoffs have resulted in word size staying the same
- Instruction Level Parallelism: Core executes instructions in parallel
- Pipelining (Time): Split instruction execution into multiple stages, so that multiple instructions can occupy different stages in the same clock cycle (if they do not have dependencies)
- Hazards might result in stalling, which can be mitigated with advanced techniques like branch prediction, out of order execution, or prefetching
- Superscalar (Space): Have more slots within each pipeline stage, so that multiple instructions can be run at the same time although it only supports one overall execution flow
- Scheduling is challenging, typically dynamic
- Structural hazards cause problems, but more instructions per cycle possible and fewer cycles per instruction
- Thread Level Parallelism: Multiple physical contexts in a physical core, threads share resources
- SMT: Simultaneous multithreading, processor support for multithreading
- Processor Level Parallelism (Multiprocessing): Have more physical cores and thus execution flows

Flynn's Taxonomy

- Instruction Stream: Single execution flow
- Data Stream: Data being manipulated by instruction stream
- SISD: Single instruction stream, each instruction operates on a single unit of data, most uniprocessors fall into here
- SIMD: Each instruction can now work on multiple data, supported by most modern processors through multiple ALUs
- MISD: Multiple instruction streams, but all working on the same data, very rare
- MIMID: Each PU has its own instruction and data, most popular model for modern multiprocessors
- Stream Processors like GPUs have a set of threads executing same code (SIMD) and multiple sets of threads executing in parallel (MIMD)

Memory Organisation

- Typically made up of core, one or more levels of caches, and memory module
- Memory Latency: Amount of time for memory request from a processor to be serviced by memory system
- Memory Bandwidth: Maximum rate at which memory system can provide data to a processor
- Processor stalls when it cannot run next instruction while waiting for memory
- Bandwidth is the critical resource and parallel programs should fetch data from memory less often by exploiting temporal locality and sharing data across threads
- Distributed Memory Systems: Each node is an independent unit, memory in a node is private and data is sent explicitly between nodes
- Shared Memory System: Parallel programs access memory through shared memory provider, and might not be aware of actual hardware memory architecture
- Cache Coherence: Multiple copies of same data exist on different caches, changes need to be propagated
- Factors to differentiate shared memory systems: Processor to Memory Delay, Presence of local cache with cache coherence protocol
- Uniform Memory Access (UMA): Latency of accessing main memory is the same for every PU, suitable for a small number of PUs which are fighting for limited bandwidth
- Non-Uniform Memory Access (NUMA): Physically distributed memory of all PUs are combined to form a global shared memory address space; accessing local memory is faster than remote memory so it becomes non-uniform
- Cache Only Memory Architecture (COMA): Each memory block works as cache memory; data migrates dynamically according to cache coherence schemes
- No need to partition code or data, or move data among processors
- Special synchronisation constructs are required; not scalable due to contention

Parallel Programming Models

- Total Work consists of work done for actual tasks and overhead of dependencies
- Data Parallelism: Partition data used in solving the problem, each PU carries out similar operations on its part of the data e.g. SIMD
- Loop parallelism: Run loop iterations in parallel if possible
- Task Parallelism: Partition tasks used to solve the problem among PUs
- Task Dependence Graph: Visualises decomposition strategies as DAG
- Critical Path Length is maximum slowest completion time
- Degree of Concurrency is Total Work / Critical Path Length
- Maximum Concurrency is maximum number of tasks running in parallel at any point in time

Models of Coordination

- Shared Address Space: Tasks communicate by reading and writing using shared variables, ensuring mutex through locks
- Requires hardware support to implement efficiently due to contention, matches shared memory systems
- Very little structure so it is simple, but hard to scale
- Message Passing: Tasks operate within their own private address space and communicate with each other by sending and receiving messages e.g. MPI
- Hardware does not implement system-wide loads and stores, matching distributed memory systems
- Highly structured communication, allowing scalability but complex to program and network communication has delay

Program Parallelisation

- Granularity of Computation: Coarse or Fine Grained
- Foster's Design Methodology (PCAM)
  - 1. Partitioning: Splitting problem into smaller tasks
  - Data Centric / Domain Decomposition: Divide data into smaller pieces, associate computations with data (data parallelism)
  - Computation Centric / Functional Decomposition: Divide computation into pieces, determine how to associate data with computations (task parallelism)
  - Rules of Thumb: 10x more primitive tasks than cores, minimise redundant computations and data storage, have primitive tasks of roughly the same size, number of tasks increasing function or problem size
  - 2. Communication: Provide data required by tasks
  - Local Communication: Task needs data from small number of other tasks
  - Global Communication: Significant number of tasks contribute data to perform a computation
  - Rules of Thumb: Keep communication operations balanced among tasks, have each task communicating with a small group of neighbours, overlap computation with communication
  - 3. Agglomeration: Decrease communication and development costs while maintaining flexibility
  - Combine tasks into larger tasks to improve performance and maintain scalability
  - Too many tasks leads to high communication overhead
  - Rules of Thumb: Increase locality of parallel algorithm, ensure tradeoff between agglomeration and code modification costs is reasonable
  - 4. Mapping: Mapping tasks to processors to minimise execution time
  - Conflict between maximising processor utilisation and minimising inter-processor communication
  - Optimal mapping is NP-hard, use some sort of heuristic

- Evaluate static and dynamic task allocation options, cores should be saturated and task allocator should not be performance bottleneck
- Hard to analyse dependencies
- Execution time is hard to predict

Parallel Programming Patterns

- Fork-Join: Task creates child tasks which run independently in parallel
- Parbegin-Parend: When execution reaches this construct, a set of threads is created
- Statements after this construct are only executed after all these threads are done
- All forks are done at the same time, all joins are done at the same time
- SIMD: Single instructions executed synchronously by different threads on different data
- SPMD: Same program executed on different cores on different data, no implicit synchronisation
- Master-Worker: Single master program controls execution of program, worker task waits for instructions from master task
- Task (Work) Pools: Threads retrieve and store tasks and execute those tasks
- Access to task pool must be synchronised to avoid race conditions; execution is completed when all threads are done and task pool is empty
- Useful for adaptive and irregular applications, but introduces synchronisation overhead
- Producer-Consumer: Producer threads produce data which are used as input by consumer threads, synchronisation is needed
- Pipelining: Data in application is processed as stream; elements flow through pipeline stages which perform different processing steps which might have uneven execution times

Performance of Parallel Systems

- User Goals: Reduced response time / latency
- Computer Managers: High throughput / work done per unit time
- Execution Time $T_p(n)$: Wall-clock time, consisting of computation, exchange of data, synchronisation, waiting time, etc.
- Speedup: $S_p(n) = T_*(n)/T_p(n)$ Ratio of best sequential execution time to parallel execution time, possible to have $S_p(n) > p$
- Cost: $C_p(n) = p \times T_p(n)$ Estimation of total amount of work done by all PUs
- Cost-Optimal: Executes same total number of operations as the fastest sequential program
- Efficiency: $E_p(n) = S_p(n)/p = T_*(n)/C_p(n)$, measure of how good a speedup is to theoretically maximum speedup
- Amdahl's Law: Speedup of parallel execution is limited by sequential fraction of code f that cannot be parallelised which forms a bottleneck, $S_p(n) = \frac{1}{f+(1-f)/p} \leq \frac{1}{f}$, estimate $f = \frac{p/S_p(n)-1}{p-1}$
- Manufacturers discouraged from making large parallel computers if most problems have large sequential fractions
- Gustafson's Law: Sequential fraction is not constant wrt problem size, this fraction shrinks as problems grow larger
- $S_p(n) = f' + (1 - f')p$, where f' is the sequential fraction corresponding to the problem of size n, derived from sequential $T_{par} = (1 - f') \times T_p(n) \times p$
- Amdahl's speedup is relative to serial-processor execution time, while Gusafson's is derived from parallel-processor execution time.

Modelling Program Performance

- Response Time (wall-clock): User CPU time, System CPU time, Waiting time
- $Time_{user}(A) = N_{instr}(A) \times CPI(A) \times Time_{avg\_cycle}$
- Depends on total number of instructions, average CPI, and average time per CPU cycle
- To consider memory, can make term $N_{cycle}(A) + N_{mm\_cycle}(A)$
- $N_{mm\_cycle}(A) = N_{read\_cycles}(A) + N_{write\_cycles}$, $N_{read\_cycles}(A) = N_{read\_ops}(A) \times R_{read\_miss\_rate}(A) \times N_{miss\_cycles}$, this can be expanded to multilevel caches
- Altogether, $Time_{user}(A) = ([N_{instr}(A) \times CPI(A)] + [N_{rw\_ops}(A) \times R_{miss\_rate}(A) \times N_{miss\_cycles}]) \times Time_{avg\_cycle}$
- Throughput can be measure with Million Instructions Per Second, $MIPS(A) = N_{instr}(A)/(Time_{user}(A) \times 10^6)$
- Another variation only measures floating point operations, MFLOPS
- Arithmetic Intensity: Amount of computation / Amount of communication
- Roofline Model: Performance against Arithmetic Intensity, increases as bound by bandwidth then levels out when bounded by peak performance
- Contention: Resources can perform operations at a given throughput, but many requests might be made to the same resource within a small window of time
- Can exploit temporal or spatial locality to reduce this
- Possible Bottlenecks and How to diagnose:
  - Instruction-rate limited: Add more math and see effect on execution time
  - Memory bottleneck: Remove math but load same data
  - Locality of data access: Arbitrarily change all array access to the same place
  - Synchronisation overhead: Remove all atomic operations and locks
  - Profile e.g. with perf then optimise

Memory Consistency

Cache Coherence

- Behaviour of reads and writes to a specific memory location
- Program Order: A single core should see its memory reads and writes in program order (if no other cores write)
- Write Propagation: Writes from a core should eventually become visible to other cores
- Transaction Serialisation: All cores should agree on the order of writes to a memory location
- Track invalid or valid states of cache blocks
- If invalid, reading forces other caches to write back first
- If valid, writing to it invalidates all other caches
- Cache Coherence Protocols are enforced by hardware cache controllers
- Snooping Based: Each cache maintains state of own cache blocks, cache controllers broadcast onto and monitor shared bus for updates
- All PUs can observe every bus transaction, bus puts events into a single order, individual PUs maintain program order
- Granularity of coherence is per cache block
- Directory Based: Central directory maintains state, no broadcasts
- CC causes overhead in shared memory systems, appearing as increased memory latency and lowering hit rate
- Cache Ping-Pong: 2 PUs keep accessing the same cache line, causing it to be repeatedly invalidated
- False Sharing: 2 PUs write to different memory addresses which share a cache line, can be avoided with padding

Memory Consistency Models

- Coherence: All PUs must agree on order of reads and writes to same memory location
- Memory Consistency: Constrains the order in which memory operations performed by one thread become visible to other threads for different memory locations
- Consistency models allow instructions within a program or core to be reordered to achieve better performance
- Memory Ordering Requirements: WR, RR, RW, WW
- Sequentially Consistent: Preserves all four, every PU issues operations in program order
- Relaxed Consistency: Relax order only if there are no data dependencies between operations (accessing same memory location), no modern CPU provides it by default
- WR relaxed consistency: Allow a read to be reordered wrt to the previous write of same processing unit, hiding latency of slow writes, provided that data dependencies are preserved and caches are coherent
- Total Store Ordering (TSO): A PU can perform a later read before its previous write is seen by all other PUs, reads by other PUs cannot return the new value of the write until it is observable by all PUs (write atomicity)
- Processor Consistency (PC): May return the value of any write, even from another PU, before the write is observed by all PUs, write atomicity is not preserved, although it is eventually propagated
- WW relaxed consistency: Writes can bypass earlier writes to different location in write buffer (non FIFO write buffer)
- Partial Store Ordering (PSO): Relaxed WR, WW, maintains write atomicity

GPGPU Programming

- Many processors are needed to maximise speedup
- Adding more nodes increases communication overhead
- Can utilise GPU instead
- CPU and GPU sit on the same motherboard, connected via high speed links such as PCIe
- Programmers used to write shaders which GPU will run in parallel
- GPGPU: Program in C++ and interface with GPU via API calls, giving more control

CUDA GPU Hardware Architecture

- CPU has a few complex cores while GPU has many simpler cores
- Each GPU thread is not as capable as a CPU thread as it focuses on instruction throughput and latency hiding to maximise parallelism
- Compute Capability indicates different features and internal architecture
- Latency Hiding: Switch to waiting threads when one stalls
- Streaming Processor (Hopper): Close to physical core, tries to execute 32 CUDA threads (warp) at the same time
- Massive register file for instant context switching
- Streaming multiprocessor consists of 4 SPs
- SPs share L1 cache which can also be used for SHM between threads
- 4 warps can execute in true parallel
- Full GPU has 144 SMs, sharing an L2 cache, all having access to slower GPU DRAM main memory

CUDA Programming Model

- `nvcc` compiler outputs CPU code and GPU intermediate assembly code (PTX / parallel thread execution)
- PTX code is transformed into low-level NVIDIA SADD code then binary
- CPU and GPU code is then linked together with CUDA libraries to be run
- Device is the GPU, Host is the CPU, Kernel is function called by host that then runs on device with specified execution configuration
- CUDA threads are organised into blocks, programmer can specify how many threads to have
- Each blocks is assigned to only 1 SM for entire kernel duration, having shared memory and synchronisation within block
- CUDA blocks are organised into a grid, programmer can specify how many blocks per grid; no synchronisation across blocks

CUDA Execution Model

- Each block is broken down into sets of 32 threads (warps)
- Single Instruction Multiple Thread (SIMT): Each of the threads in the warp starts at the same PC, executing in lockstep as far as possible

- Warp scheduler decides how to schedule warps with immediate context switching
- Programmer code could cause threads to diverge and result in lower instruction throughput

CUDA Memory Model

- Each thread has up to 255 registers; local memory (GPU DRAM) is used if data does not fit
- Every block has shared memory
- All can access slow but cached GPU DRAM
- Consists of global memory, constant memory (R, linear), texture memory (R, 2D spatial)
- CUDA's unified memory model avoids the need for manual copying of data between host and device
- API allows for synchronisation using barriers

Optimising CUDA Programs

- Optimise memory usage to achieve maximum memory bandwidth
  - Minimise data transfer between host and device
  - Minimise global memory accesses by using SHM
  - Ensure global memory access is coalesced whenever possible
  - When warp accesses memory, requests are combined into requests of 32 bytes each since that is the size of a request from global memory
  - Minimise bank conflicts during SHM access
  - Banks are faster and have higher bandwidths than global memory, divided into equally sized memory modules
  - Bank conflicts are when two addresses of a memory request fall into the same memory bank and access has to be serialised
  - Overlap asynchronous transfers with computation where possible
- Maximise parallel execution to increase data parallelism and occupancy / utilisation
  - Occupancy: Number of active warps / Maximum possible
  - Low occupancy makes it impossible to hide memory latency
  - Too much latency might cause too much data usage and slow local memory usage
  - Using more blocks allows more SMs to run in parallel
- Optimise instruction usage to achieve maximum instruction throughput and avoid branching
  - Minimise use of arithmetic instructions with low throughput
  - Trade precision for speed by avoiding integer divison and modulo
  - Minimise warp divergence

Message Passing and Distributed Address Space

- Hard to have shared address space with multiple machines and distributed memory
- Distributed address space has data explicitly partitioned for each process
- All interaction requires both parties to participate
- Message Passing: Data exchanged via commuincation operations
- Message Passing Interface is a standardised specification for message passing libraries, provided as a set of library calls
- Processes can be on different nodes, they do not share any memory
- Tasks only synchronise to communicate, they run independently

Point to Point Communication

- Blocking: Call only returns when resources used in call may be reused safely by programmer
- For non-blocking, use test to check status of call
- MPI Messages consists of data and envelope (routing)
- Data consists of start address, count, and datatype
- Envelope consists of source or destination (process rank), tag (arbitrary number), communicator (set of processes being communicated with)
- Non-overtaking property: If a sender sends two messages to the same receiver, they are delivered in order of sending
- Deadlock can occur when message passing cannot be completed
- Buffered: User provided data is copied into internal buffer and control is returned to user
- Memory copy speed is much faster than network speed
- Without system buffers, blocking sends must wait for other side to receive data before returning
- Synchronous: Send can complete only when a matching receive has started to execute, requires coordination with other processes
- Asynchronous: Send can complete regardless of communication with other processes
- Non Buffered, Blocking: Considerable idling overheads due to mismatches in timing
- Buffered, Blocking: Copying to internal buffer removes idling time; without hardware support, receiver is interrupted to transfer data to own buffer
- Non Buffered, Non Blocking: Cannot reuse data until done but no idling time

Process Groups and Communicators

- Process Group: Ordered set of processes, each with a unique rank within
- Processes can be members of multiple groups
- Communicator: Handle that processes can use to communicate with each other
- Intra: Point to point and collective communication operations on a single group; Inter: Between two process groups
- Virtual Topologies: Logical organisations of processes, allowing neighbours to be easily addressable
- Communicators with grid or graph style of addressing process ranks

Collective Communication

- Operations involving ALL processes in a communicator
- Barriers are the only collective synchronisation operation; all processes in communicator must execute the function call
- Single Broadcast: Sender sends same block of data to all other processes; every process must call it
- Multi-Broadcast: Every processor sends a data block to every other processor, collected in rank order
- Scatter: Root process divides data among itself and others; Gather: Root process collects data
- Single-Accumulation: Each processor provides a block of data with the same type and size before reduction operation (binary, associative, commutative) is applied element by element, with results placed in root processor
- Multi-Accumulation: Each processor provides for every other processor a potentially different data block
- Total Exchange: Each processor provides for each other processor a potentially different data block

Interconnection Networks and Data Distribution

Data Distribution

- Data is usually found in arrays, figure out how to decompose them for distribution and maximise data parallelism
- Blockwise: Allocate contiguous block to each processor, good for spatial locality
- Cyclic: Take only multiples for each processor, good for balancing load
- For 2D arrays, can combine them across dimensions
- Block-Cyclic: Form blocks then perform cyclic allocation, looks like wider columns
- Checkerboard: Processors organised into 2D mesh, can then be applied blockwise, cyclic, or both

Interconnection Networks

- How different components in the system are connected
- Ring: Linear connections which loopback
- Torus: 2D mesh with wrapping
- Shear Sort:
  - For a 2D mesh
  - Odd rows sort ascending, even rows sort descending
  - All columns sort in ascending order
  - Repeat until sorted into snake order
  - Log phases needed, giving sqrt x log complexity

Interconnect Topology

- Direct / Static / Point-to-Point Interconnection: Nodes directly connected, often of same type
  - Topology can be modeled as graph
  - Diameter: Maximum distance between any pair of nodes
  - Degree: Number of direct neighbours of a node
  - Bisection Width: Minimum number of edges removed to cut network into equal halves, measure of worst case network capacity
  - Edge / Node Connectivity: Minimum number removed to disconnect network
  - Cube Connected Cycles: Replace each vertex in a hypercube with a cycle; node has index in hypercube and in cycle
  - Dragonfly: Groups of all-to-all connected switches, all groups all-to-all connected, reduces diameter
  - Fat Tree: Tree with more edges (bandwidth) closer to root nodes
- Indirect / Dynamic Interconnection: Interconnect is formed by switches rather than direct links
  - Reduces hardware costs, can connect different types of hardware
  - Bus Network: Set of wires to transport data from sender to receiver, only one pair of devices can communicate at a time via bus arbiter
  - Multistage Switching Network: Several intermediate switches with connecting wires between neighbouring stages
  - Crossbar Network: Literally grid network where each node is either in a straight or direction change state
  - Omega Network: One unique path for every input to output, $\log n$ stages total, $n/2$ switches per stage, connections between stages are regular, known as $\log n - 1$ dimension omega network
  - For each layer, connect to first inputs first, then second inputs
  - For a switch at position α in stage i, there is an edge from (α, i) to $(\beta, i+1)$ where $\beta = \alpha$ with a cyclic left shift (and inversion of LSBit)
  - Butterfly Network: (α, i) connects to $(\alpha, i+1)$ and $(\alpha', i+1)$ if they differ in the $(i+1)$th bit from the left
  - Baseline Network: Connect to $(\beta, i+1)$ where it is a cyclic right shift of the last $(k-i)$ bits of α where k is the number of bits, or inversion of LSBit of α followed by cyclic right shift of last $(k-i)$ bits

Routing

- Use routing algorithms
- Minimal: Whether shortest path is always chosen
- Adaptivity: Deterministic v/s Adaptive (takes into account network status)
- XY Routing: Move in X direction until match, then in Y direction
- E-Cube Routing for Hypercube: Number of hops is hamming distance, match from MSB to LSB or other way round
- XOR-Tag Routing for Omega Network: Let $T = S \oplus D$, then for stage k, go straight if bit k of T is 0, crossover otherwise

Energy Efficient Computing

- Energy is measured in Joules (J)

- Power (P), measure in Watts / J/s: Amount of energy transferred or converted per unit time
- Higher power processors typically give us more performance
- Most power used by computers is converted into heat
- Dennard Scaling: Smaller transistors should use the same power per unit area, ended about 2005
- Power Wall: Cannot increase frequency and cool at high power densities

Per-Processor Efficiency

- Performance per Watt based on fixed benchmark program
- Not constant as processors can operate at different power levels, one factor is clock frequency
- Clock frequency results in much more power usage increase compared to score increase
- Processor Voltage has a much more significant effect on processor power and temperature, higher frequencies require more than linear voltage increases
- $P_{total} = P_{dynamic} + P_{static}$, $P_{dynamic} = kV^2f$
- Dynamic Voltage and Frequency Scaling: Adjusting voltage and clock frequency dynamically to reduce power usage and heat
- Heterogeneous Cores: Two different processors side by side e.g. P-cores (efficient at high performance) and E-cores (efficient at low performance)

Datacenter/HPC Efficiency

- Easy to access and users don't have to manage
- GFLOPs-per-watt, similar to per-processor efficiency
- Power Usage Effectiveness: Energy used for compute v/s Total energy usage
- Hopper Superchip: High bandwidth CPU-GPU connection
- $PUE = \frac{TotalEnergy}{ITEnergy} = 1 + \frac{NonITEnergy}{ITEnergy}$
- Hotter climates at disadvantage
- Running larger compute loads often decreases PUE
- Aisle Containment: Hot aisle cold aisle, point hot air sides together for more efficient cooling
- Warm-Water Cooling: Use warm water to cool machines

Topology	Degree (g)	Diameter (δ)	Edge Conn. (ec)	Bisection (B)
Complete	$n-1$	1	$n-1$	$(n/2)^2$
Linear Array	2	$n-1$	1	1
Ring	2	$\lfloor n/2 \rfloor$	2	2
d-dim Mesh ($n = r^d$)	$2d$	$d(n^{1/d}-1)$	d	$n^{(d-1)/d}$
d-dim Torus ($n = r^d$)	$2d$	$d\lfloor n^{1/d}/2 \rfloor$	$2d$	$2n^{(d-1)/d}$
k-ary d-cube ($n = k^d$)	$2d$	$d\lfloor k/2 \rfloor$	$2d$	$2k^{d-1}$
Comp. Bin. Tree ($n = 2^k - 1$)	3	$2(k-1)$	1	1
k-dim Hypercube ($n = 2^k$)	k	k	k	$n/2$
CCC ($n = k \cdot 2^k$)	3	$2k - 1 + \lfloor k/2 \rfloor$	3	2^{k-1}